Light

Data Representation — Numbers

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Number bases
2. Converting between bases
3. Binary arithmetic
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Data Representation: Numbers (8525 Paper 2)**. This topic covers number bases (binary, decimal, hexadecimal), conversions between bases, units of information, binary arithmetic, and binary shifts.
Exam questions typically ask you to **convert** between number bases, **add binary numbers**, **apply binary shifts**, and **explain** why computers use binary and hexadecimal. Expect questions worth 1—4 marks requiring clear, stepwise working. AQA uses **decimal prefixes** (1 kB = 1000 bytes), not binary prefixes.

Why data representation matters

Computers can only process binary — patterns of 0s and 1s. Everything you see, hear, or interact with on a computer (text, images, sound, videos, programs) must ultimately be represented in binary. Understanding number systems and how to convert between them is fundamental to understanding how computers work.

- **Binary (base 2):** The only number system computers understand internally — two states (0 and 1) map directly to electronic switches being off or on
- **Hexadecimal (base 16):** A shorthand for binary that makes long binary strings easier for humans to read and work with
- **Decimal (base 10):** The number system humans use in everyday life

In the exam, you must show clear working for all conversions and calculations. AQA expects step-by-step methods, not just final answers.

What you must know

This checklist covers the key exam-visible concepts from the AQA specification. You should be able to explain each point and demonstrate each skill.

- **Understand** the number bases: decimal (base 10), binary (base 2), and hexadecimal (base 16).
- **Explain** why computers use binary to represent all data and instructions.
- **Explain** why hexadecimal is often used in computer science.
- **Convert** between binary, decimal, and hexadecimal (values 0—255).
- **Know** that a bit is the fundamental unit and a byte is 8 bits.
- **Know** the decimal prefixes: kilo (1000), mega (1000 kB), giga (1000 MB), tera (1000 GB).
- **Add** up to three 8-bit binary numbers together.
- **Apply** binary shifts (left and right) and describe their effect on values.

> **Specification note.** AQA 8525 assesses data representation on Paper 2 (Computing concepts). Maximum values for conversions are: decimal 255, binary 1111 1111, hexadecimal FF. Binary arithmetic uses a maximum of 8 bits with no carry beyond the 8th bit. Only logical binary shifts are required.

Toolkit

Powers of 2 (essential for conversions)

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	**64**	**32**	**16**	**8**	**4**	**2**	**1**

Memorise this row! It is essential for binary-to-decimal and decimal-to-binary conversions.

Hexadecimal digits

Decimal	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Hex	0	1	2	3	4	5	6	7	8
Binary	0000	0001	0010	0011	0100	0101	0110	0111	100

Each hexadecimal digit represents exactly 4 bits (a nibble). This is why hex is so useful for representing binary.

Units of information (decimal prefixes)

Unit	Symbol	Value
Bit	b	Fundamental unit (0 or 1)
Byte	B	8 bits
Kilobyte	kB	1,000 bytes
Megabyte	MB	1,000 kB = 1,000,000 bytes
Gigabyte	GB	1,000 MB = 1,000,000,000 bytes
Terabyte	TB	1,000 GB = 1,000,000,000,000 bytes

Exam tip: AQA uses **decimal prefixes** (1 kB = 1000 bytes), NOT binary prefixes (1 KiB = 1024 bytes). Always use 1000 as the multiplier in your calculations unless told otherwise. This is a common source of lost marks.

Maximum values for 8 bits

For AQA GCSE, you work with numbers up to 8 bits (one byte). These are the key values to remember:

- **Maximum decimal value:** 255
- **Maximum binary value:** 1111 1111
- **Maximum hexadecimal value:** FF
- **Minimum value:** 0 (in all bases: 0, 0000 0000, 00)
- **Number of possible values:** 256 (from 0 to 255 inclusive)

With n bits, you can represent 2^n different values (0 to $2^n - 1$). So 8 bits gives 2^8 = 256 values.

1. Number bases

A number base (or radix) determines how many different digits are available and what each position in a number represents. Humans typically use base 10 (decimal) because we have 10 fingers. Computers use base 2 (binary) because electronic circuits have two states: on (1) and off (0). Hexadecimal (base 16) provides a convenient shorthand for binary that is easier for humans to read and work with.

Understanding how different number bases work is fundamental to understanding how computers store and process data. Every piece of information in a computer — from text and images to programs and calculations — is ultimately represented as patterns of binary digits.

1.1 Decimal (base 10)

Decimal is the number system we use every day. It has 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. The name comes from the Latin word "decem" meaning ten.

Each position represents a power of 10. The rightmost digit is the ones (10^0), the next is tens (10^1), then hundreds (10^2), and so on. For example, in the number 352:

- 3×100 (10^2) = 300
- 5×10 (10^1) = 50
- 2×1 (10^0) = 2
- Total = 352

We are so familiar with decimal that we rarely think about place values, but understanding this concept is essential for working with other number bases.

1.2 Binary (base 2)

Binary has only 2 digits: 0 and 1. Each digit is called a **bit** (binary digit). A group of 8 bits is called a **byte**, and a group of 4 bits is called a **nibble**.

Each position represents a power of 2. The rightmost digit is $2^0 = 1$, then $2^1 = 2$, then $2^2 = 4$, and so on. For example, in the binary number 1011:

- 1×8 (2^3) = 8
- 0×4 (2^2) = 0
- 1×2 (2^1) = 2
- 1×1 (2^0) = 1
- Total = 11 (in decimal)

An 8-bit binary number can represent values from 0 (00000000) to 255 (11111111). This is the range you need to know for AQA GCSE.

1.3 Why computers use binary

Computers use binary because their fundamental components — transistors — work as electronic switches that are either **on** or **off**. This two-state system maps perfectly to binary:

- **On** (high voltage, typically 5V or 3.3V) = 1
- **Off** (low voltage, typically 0V) = 0

Using only two states has significant advantages for reliability. If circuits had to distinguish between 10 different voltage levels (for decimal), small fluctuations or electrical noise could cause errors. With only two states that are far apart, the system is much more robust and reliable.

All data and instructions in a computer are represented using binary. This includes:

- **Text:** Each character is represented by a binary code (e.g., ASCII or Unicode)
- **Images:** Each pixel's colour is stored as binary values
- **Sound:** Audio samples are stored as binary numbers
- **Programs:** All instructions that tell the computer what to do are binary
- **Numbers:** All calculations are performed in binary

A bit pattern could represent different types of data depending on how the computer interprets it. The same binary pattern might be a number, a character, part of an image, or a machine instruction — context determines the meaning.

> **Exam tip:** When asked "what do computers represent using binary?", the correct answer is "all data AND instructions". Not just data, not just some things — everything. This is a very common exam question.

1.4 Hexadecimal (base 16)

Hexadecimal (often shortened to "hex") has 16 digits: 0—9 and A—F (where A=10, B=11, C=12, D=13, E=14, F=15). The letters can be uppercase or lowercase in exams.

Each hexadecimal digit represents exactly 4 binary digits (bits). This is not a coincidence — $16 = 2^4$, so there are 16 possible combinations of 4 bits, which maps perfectly to 16 hex digits. This relationship makes hex a convenient shorthand for binary:

- Binary: 1111 0011 (8 digits, easy to misread)
- Hex: F3 (2 digits, much clearer)

Two hex digits can represent one byte (8 bits), so a byte can have values from 00 to FF in hexadecimal, which equals 0 to 255 in decimal.

1.5 Why hexadecimal is used in computer science

Hexadecimal is used because it provides a **compact, human-readable representation** of binary data. The key advantages are:

- **Easier to read:** Long binary strings are hard for humans to read without making errors. A 32-bit binary number would be 32 digits long, but only 8 hex digits.
- **Easy to convert:** Each hex digit maps to exactly 4 bits, making conversion straightforward. You can convert in your head once you know the mappings.
- **Reduces errors:** Shorter representations mean fewer opportunities for transcription errors.
- **Standard in industry:** Many computer systems and programming tools display values in hexadecimal by default.

Common uses of hexadecimal include:

- **Memory addresses:** Locations in computer memory are typically shown in hex (e.g., 0x7FFF5FBFF8C0)
- **Colour codes:** Web colours use hex (e.g., #FF0000 for red, #00FF00 for green, #0000FF for blue)
- **MAC addresses:** Network hardware addresses (e.g., 00:1A:2B:3C:4D:5E)
- **Error codes:** System errors are often displayed in hex
- **Machine code:** Low-level instructions are often represented in hex

Worked Example 1 — Explaining why computers use binary

Question: Explain why computers use binary to represent data and instructions. [2 marks]

Model answer:
Computers use electronic switches (transistors) that can only be in two states: on or off **(1)**.
These two states map directly to binary digits: on = 1 and off = 0 **(1)**.

Exam tip: Focus on the physical reason (two-state electronic switches) and the mapping to binary. Do not just say "it's easier" — explain WHY.

Worked Example 2 — Explaining why hexadecimal is used

Question: Explain why hexadecimal is often used in computer science. [2 marks]

Model answer:
Hexadecimal provides a more compact/shorter representation of binary data, making it easier for humans to read **(1)**.
Each hexadecimal digit represents exactly 4 binary digits, making conversion between hex and binary straightforward **(1)**.

Exam tip: AQA mark schemes accept: easier to read, fewer digits needed, less chance of error, easier to convert to/from binary.

Worked Example 3 — Identifying what binary represents

Question: Which statement best describes what computers represent using binary? [1 mark]
A: All data are represented using binary.
B: All data and instructions are represented using binary.
C: Some data and instructions are represented using binary.
D: Some instructions are represented using binary.

Model answer:
B: All data and instructions are represented using binary.
Computers can only process binary, so everything — including the instructions that tell the computer what to do — must be in binary.

Exam tip: This is a common MCQ. Remember: ALL data AND instructions. Not "some", not just "data".

Worked Example 4 — Understanding place values

Question: State the decimal value of the binary number 10000000. [1 mark]

Model answer:
10000000 in binary = 128 in decimal.
The leftmost bit represents 2^7 = 128. All other bits are 0.

Exam tip: Know your powers of 2! 2^7 = 128 is the largest value in an 8-bit number when only one bit is set.

2. Converting between number bases

You must be able to convert between binary, decimal, and hexadecimal in both directions. AQA expects values between 0 and 255 (which is 0000 0000 to 1111 1111 in binary, or 00 to FF in hexadecimal). This range corresponds to one byte of data.

There are six possible conversions between three number bases. Each conversion has a standard method that you should learn and practise until it becomes automatic. Showing your working is essential — even for 1-mark questions, writing the method helps you avoid errors and can earn partial credit if you make a slip.

2.1 Binary to decimal

Method: Write out the place values (128, 64, 32, 16, 8, 4, 2, 1) above each bit, then add up the values where there is a 1. This is the most common conversion in exams.

Example: Convert 10110111 to decimal.

128	64	32	16	8	4	2	1
1	0	1	1	0	1	1	1

Add the values with 1s: 128 + 32 + 16 + 4 + 2 + 1 = **183**

Common patterns to recognise:

- 11111111 = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255 (maximum 8-bit value)
- 10000000 = 128 (just the leftmost bit)
- 01111111 = 127 (all bits except leftmost)
- 00000001 = 1 (just the rightmost bit)

2.2 Decimal to binary

Method (subtraction): Start with the largest power of 2 that fits into your number. Subtract it and repeat with the remainder. Put a 1 in each position used, 0 elsewhere.

Example: Convert 200 to binary.

- 200 ≥ 128? Yes. 200 − 128 = 72. Write **1** in 128 column
- 72 ≥ 64? Yes. 72 − 64 = 8. Write **1** in 64 column
- 8 ≥ 32? No. Write **0** in 32 column
- 8 ≥ 16? No. Write **0** in 16 column
- 8 ≥ 8? Yes. 8 − 8 = 0. Write **1** in 8 column
- 0 ≥ 4? No. Write **0** in 4 column
- 0 ≥ 2? No. Write **0** in 2 column
- 0 ≥ 1? No. Write **0** in 1 column

Answer: 200 in decimal = **11001000** in binary

Check: 128 + 64 + 8 = 200 ✓

Alternative method (repeated division by 2): Divide by 2 repeatedly, noting the remainders. Read the remainders from bottom to top. This method is more mechanical but can be useful for larger numbers.

2.3 Binary to hexadecimal

Method: Split the binary number into groups of 4 bits (nibbles), starting from the right. If needed, pad with leading zeros on the left to complete the leftmost group. Convert each group to its hex digit using the conversion table.

Example: Convert 10110111 to hexadecimal.

- Split into nibbles: **1011** | **0111**
- 1011 = 8 + 2 + 1 = 11 = **B**
- 0111 = 4 + 2 + 1 = 7 = **7**

Answer: 10110111 in binary = **B7** in hexadecimal

Key conversions to memorise: 1111 = F, 1010 = A, 1100 = C, 0000 = 0. Knowing these makes the process faster.

2.4 Hexadecimal to binary

Method: Convert each hex digit to its 4-bit binary equivalent, then join them together. This is the reverse of binary to hex and is very straightforward once you know the digit mappings.

Example: Convert 5E to binary.

- 5 in hex = 0101 in binary (4 + 1 = 5)
- E in hex = 14 in decimal = 1110 in binary (8 + 4 + 2 = 14)

Answer: 5E in hexadecimal = **01011110** in binary

Important: Always write exactly 4 bits for each hex digit, including leading zeros. 5 must be written as 0101, not just 101, to maintain correct alignment.

2.5 Decimal to hexadecimal

Method 1 (via binary): Convert decimal → binary → hexadecimal. This uses methods you already know and is often easier for smaller numbers.

Method 2 (direct division by 16): Divide by 16 repeatedly, noting remainders. Read remainders bottom-to-top. This is more efficient for larger numbers.

Example: Convert 112 to hexadecimal (using Method 2).

- 112 ÷ 16 = 7 remainder **0**
- 7 ÷ 16 = 0 remainder **7**

Reading remainders bottom-to-top: **70**

Answer: 112 in decimal = **70** in hexadecimal

Check: 7 × 16 + 0 × 1 = 112 ✓

Example: Convert 200 to hexadecimal (using Method 2).

- 200 ÷ 16 = 12 remainder **8**
- 12 ÷ 16 = 0 remainder **12 = C**

Reading remainders bottom-to-top: **C8**

Answer: 200 in decimal = **C8** in hexadecimal

Check: 12 × 16 + 8 × 1 = 192 + 8 = 200 ✓

2.6 Hexadecimal to decimal

Method: Multiply each digit by its place value (16^1 = 16 for the left digit, 16^0 = 1 for the right digit in a 2-digit hex number) and add the results together.

Example: Convert A3 to decimal.

- A = 10, 3 = 3
- (10 × 16) + (3 × 1) = 160 + 3 = **163**

Answer: A3 in hexadecimal = **163** in decimal

Example: Convert FF to decimal.

- F = 15, F = 15
- (15 × 16) + (15 × 1) = 240 + 15 = **255**

Answer: FF in hexadecimal = **255** in decimal (maximum 8-bit value)

Worked Example 5 — Binary to decimal conversion

Question: Convert the binary number 10110111 into decimal. [1 mark]

Model answer:
Place values: 128 + 32 + 16 + 4 + 2 + 1 = **183**

Exam tip: Even for 1-mark questions, writing out the place values helps avoid errors. Write: 128+32+16+4+2+1=183.

Worked Example 6 — Decimal to hexadecimal conversion

Question: Convert the decimal number 112 into hexadecimal. Show your working. [2 marks]

Model answer:
Method: Divide by 16 repeatedly.
112 ÷ 16 = 7 remainder 0 **(1)**
7 ÷ 16 = 0 remainder 7
Reading remainders: **70 (1)**

Exam tip: "Show your working" means you MUST show the method. Without working, you may only get 1 mark.

Worked Example 7 — Hexadecimal to binary conversion

Question: Convert the hexadecimal number 9F to binary. [1 mark]

Model answer:
9 = 1001, F = 1111

9F = **10011111**

Exam tip: Memorise the 4-bit binary for each hex digit. F = 1111 is the most common to appear in exams.

Worked Example 8 — Decimal to binary conversion

Question: Convert the decimal number 235 to binary. [2 marks]

Model answer:
Step 1: Find which powers of 2 sum to 235:
235 = 128 + 64 + 32 + 8 + 2 + 1 **(1)**
Step 2: Write 1s in those positions:

128	64	32	16	8	4	2	1
1	1	1	0	1	0	1	1

Answer: **11101011 (1)**

Exam tip: Always check your answer by adding the place values back up. 128+64+32+8+2+1 = 235. ✓

3. Binary arithmetic and shifts

AQA expects you to add up to three 8-bit binary numbers together and apply logical binary shifts. All answers stay within 8 bits — there is no overflow beyond the 8th bit in GCSE questions. These skills are frequently tested and require careful, methodical working.

Binary arithmetic follows the same principles as decimal arithmetic, but with only two digits (0 and 1). The key difference is that "carrying" happens more frequently because columns fill up quickly.

3.1 Binary addition rules

Binary addition follows simple rules. Learn these until they are automatic:

- **0 + 0 = 0** (no carry)
- **0 + 1 = 1** (no carry)
- **1 + 0 = 1** (no carry)
- **1 + 1 = 10** (write 0, carry 1)
- **1 + 1 + 1 = 11** (write 1, carry 1)

Think of it like decimal: when you add 5 + 5, you get 10 — you write 0 and carry 1. In binary, 1 + 1 = 2, but 2 in binary is "10" — so you write 0 and carry 1.

When adding three numbers with carries, a column can have at most 3 ones plus a carry of 1, giving a maximum of 4. In binary: 4 = 100 (write 0, carry 10).

Work from right to left (least significant bit first), just like decimal addition. When a column sums to 2 or more, you write the remainder and carry to the next column.

3.2 Adding two binary numbers

Adding two binary numbers is the most common binary arithmetic question. Work column by column from right to left, keeping track of any carries.

Example: Add 00101101 and 00011010.

	1	1	1					Carry
	0	0	1	0	1	1	0	1
+	0	0	0	1	1	0	1	0
=	0	1	0	0	0	1	1	1

Working through each column from right to left:

- Column 1: 1 + 0 = 1
- Column 2: 0 + 1 = 1
- Column 3: 1 + 0 = 1
- Column 4: 0 + 1 = 1
- Column 5: 1 + 1 = 10 (write 0, carry 1)
- Column 6: 1 + 0 + 1(carry) = 10 (write 0, carry 1)
- Column 7: 0 + 0 + 1(carry) = 1
- Column 8: 0 + 0 = 0

Answer: 00101101 + 00011010 = **01000111**

Check in decimal: 45 + 26 = 71 ✓

3.3 Adding three binary numbers

When adding three numbers, a column can have a maximum of three 1s (plus any carry from the previous column). AQA states that students only need to add a maximum of three 1s in a single column, which gives a maximum sum of 3 (or 4 with a carry). In binary:

- 0 = 0, no carry
- 1 = 1, no carry
- 2 = 10, write 0, carry 1
- 3 = 11, write 1, carry 1
- 4 = 100, write 0, carry 10 (this only happens with 3 ones plus a carry)

Example: Add 00001010, 10011010, and 00110001.

		1	1	1		1			Carry
	0	0	0	0	1	0	1	0	
	1	0	0	1	1	0	1	0	
+	0	0	1	1	0	0	0	1	
=	1	1	0	1	0	1	0	1	

Answer: **11010101**
Check in decimal: 10 + 154 + 49 = 213 ✓

Exam tip: Always write your carry digits clearly above each column. If you make an error, the examiner can see your method and may award partial marks. Write carries in a different colour or smaller if it helps keep them distinct.

3.4 Binary shifts

A **binary shift** moves all bits left or right by a specified number of positions. AQA only requires logical shifts (not arithmetic shifts). Logical shifts are simpler: empty positions are always filled with zeros.

Left shift:

- Move all bits to the left by n positions
- Fill empty positions on the right with 0s
- Bits that move past the leftmost position are lost (discarded)
- **Effect:** Multiplies by 2^n (doubles for each position shifted)
  Example: Left shift by 1 multiplies by 2. Left shift by 2 multiplies by 4. Left shift by 3 multiplies by 8.

Right shift:

- Move all bits to the right by n positions
- Fill empty positions on the left with 0s
- Bits that move past the rightmost position are lost (discarded)
- **Effect:** Divides by 2^n (integer division, remainder discarded)
  Example: Right shift by 1 divides by 2. Right shift by 2 divides by 4. Right shift by 3 divides by 8.

Example: Apply a left shift of 2 to 00110011.

- Original: **0 0** 1 1 0 0 1 1 (decimal 51)
- Shift left by 2: The two leftmost bits (00) are discarded
- Two new zeros are added on the right
- Result: 1 1 0 0 1 1 **0 0** = **11001100** (decimal 204)
  Check: $51 \times 2^2 = 51 \times 4 = 204$ ✓

Example: Apply a right shift of 1 to 00110011.

- Original: 0 0 1 1 0 0 1 **1** (decimal 51)
- Shift right by 1: The rightmost bit (1) is discarded
- One new zero is added on the left
- Result: **0** 0 0 1 1 0 0 1 = **00011001** (decimal 25)
  Check: 51 ÷ 2 = 25.5, but integer division gives 25 (remainder discarded) ✓

Example: Apply a right shift of 3 to 01011000.

- Original: 0 1 0 1 1 **0 0 0** (decimal 88)

- Shift right by 3: The three rightmost bits (000) are discarded
- Three new zeros are added on the left
- Result: **0 0 0** 0 1 0 1 1 = **00001011** (decimal 11)

  Check: 88 ÷ 2^3 = 88 ÷ 8 = 11 ✓

Why binary shifts are useful: Binary shifts provide a very fast way to multiply or divide by powers of 2. Processors can perform shifts much faster than general multiplication or division operations because shifts are simply moving bits around — no actual calculation is needed. This makes shifts useful for optimising programs that need to multiply or divide by 2, 4, 8, 16, etc.

3.5 Limitations of binary shifts

Binary shifts can only multiply or divide by powers of 2 (2, 4, 8, 16, 32, etc.). They cannot be used to multiply by other numbers like 3, 5, or 7.

When bits are shifted out, they are lost. For right shifts, the remainder of the division is discarded (integer division only).

Worked Example 9 — Adding three binary numbers

Question: Add together the following three binary numbers and give your answer in binary. [2 marks]

```
00001010
10011010
00110001
```

Model answer:

Working from right to left, adding each column:

Column 1 (rightmost): 0+0+1 = 1

Column 2: 1+1+0 = 10 (write 0, carry 1)

Column 3: 0+0+0+1(carry) = 1

Column 4: 1+1+0 = 10 (write 0, carry 1)

Column 5: 0+1+1+1(carry) = 11 (write 1, carry 1)

Column 6: 0+0+1+1(carry) = 10 (write 0, carry 1)

Column 7: 0+0+0+1(carry) = 1

Column 8: 0+1+0 = 1

Answer: 11010101 (2)

Exam tip: Write the carry digits above each column clearly. Examiners can give partial credit if they can see where you made an error.

Worked Example 10 — Left binary shift

Question: State the result of applying a left binary shift of two to the bit pattern 00110011. [1 mark]

Model answer:
Shift all bits 2 positions left, fill with 0s on right:
00110011 → **11001100**

Exam tip: For left shifts, the first n bits are lost and n zeros are added on the right.

Worked Example 11 — Binary shift and decimal result

Question: The decimal equivalent of 00110011 is 51. State the result of applying a left binary shift of one to this bit pattern. Give your answer in decimal. [1 mark]

Model answer:
Left shift of 1 multiplies by $2^1 = 2$.
51 × 2 = **102**

Exam tip: If you're given the decimal value, you can multiply/divide directly by the power of 2 instead of performing the shift in binary.

Worked Example 12 — Describing when binary shifts are used

Question: Which statement best describes where a single binary shift can be used? [1 mark]
A: Multiply or divide numbers by any even number.
B: Multiply or divide numbers by any number.
C: Multiply or divide numbers by any odd number.
D: Multiply or divide numbers by a power of two.

Model answer:
D: Multiply or divide numbers by a power of two.
A left shift of n positions multiplies by 2^n. A right shift of n positions divides by 2^n.

Exam tip: Powers of 2 are: 1, 2, 4, 8, 16, 32, 64, 128, etc. Shifts ONLY work for these values, not "any even number".

Worked examples summary

This topic includes 12 worked examples covering all key areas tested by AQA. Use these as templates for structuring your own exam answers.

Example	Topic	Question type	Marks
1	Why computers use binary	Explain	2
2	Why hexadecimal is used	Explain	2
3	What binary represents	MCQ	1
4	Binary place values	State	1
5	Binary to decimal conversion	Convert	1
6	Decimal to hexadecimal conversion	Convert with working	2
7	Hexadecimal to binary conversion	Convert	1
8	Decimal to binary conversion	Convert with working	2
9	Adding three binary numbers	Calculate	2
10	Left binary shift	State	1
11	Binary shift (decimal result)	State	1
12	When binary shifts are used	MCQ	1

Quick reference

Number base summary

Number base	Also called	Digits used	Used for
Binary (base 2)	Base 2	0, 1	All data and instructions inside computers
Decimal (base 10)	Denary, base 10	0—9	Human everyday use
Hexadecimal (base 16)	Hex, base 16	0—9, A—F	Compact representation of binary (memory addresses, colour codes)

Conversion methods

Conversion	Method
Binary → Decimal	Write place values (128, 64, 32, 16, 8, 4, 2, 1). Add values where there is a 1.
Decimal → Binary	Subtract largest power of 2 that fits. Repeat. Put 1s in those positions.

Conversion	Method
Binary → Hex	Split into groups of 4 bits from right. Convert each group to hex digit.
Hex → Binary	Convert each hex digit to 4-bit binary. Join together.
Decimal → Hex	Divide by 16 repeatedly. Note remainders. Read bottom-to-top.
Hex → Decimal	Multiply each digit by its place value (16, 1). Add results.

Binary arithmetic rules

Operation	Result	Carry
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	1
1 + 1 + 1	1	1

Binary shift effects

Shift	Effect	Example
Left shift by 1	Multiply by 2	00000101 (5) → 00001010 (10)
Left shift by 2	Multiply by 4	00000101 (5) → 00010100 (20)
Left shift by 3	Multiply by 8	00000101 (5) → 00101000 (40)
Right shift by 1	Divide by 2 (integer)	00010100 (20) → 00001010 (10)
Right shift by 2	Divide by 4 (integer)	00010100 (20) → 00000101 (5)

Key definitions for the exam

Term	Definition (mark-scheme style)
Bit	The fundamental unit of information; a single binary digit (0 or 1).
Byte	A group of 8 bits.
Binary	Base 2 number system using only 0 and 1.
Decimal	Base 10 number system using digits 0—9.

Term	Definition (mark-scheme style)
Hexadecimal	Base 16 number system using 0—9 and A—F.
Binary shift	Moving all bits left or right by a specified number of positions.
Left shift	Moving bits to the left; multiplies by powers of 2.
Right shift	Moving bits to the right; divides by powers of 2 (integer division).

Typical pitfalls

Pitfall 1 — Using 1024 instead of 1000.
Wrong: "1 kB = 1024 bytes."
Right: AQA uses decimal prefixes: 1 kB = 1000 bytes. Always use 1000 in calculations.

Pitfall 2 — Forgetting to show working.
Wrong: Just writing the final answer for a 2-mark question.
Right: Show the method clearly. For conversions, show the place values or division steps.

Pitfall 3 — Mixing up left and right shifts.
Wrong: "Right shift multiplies by 2."
Right: Left shift multiplies by powers of 2. Right shift divides by powers of 2.

Pitfall 4 — Forgetting carry digits in binary addition.
Wrong: Adding 1+1 as 2.
Right: 1+1 = 10 in binary (write 0, carry 1). Write carry digits clearly above each column.

Pitfall 5 — Not grouping bits correctly for hex conversion.
Wrong: Splitting 11010110 as 110 10110.
Right: Always split from the RIGHT: 1101 0110. Each group must be exactly 4 bits.

Pitfall 6 — Saying "data" instead of "data and instructions".
Wrong: "Computers represent all data using binary."
Right: Computers represent all data AND instructions using binary.

Pitfall 7 — Confusing hex digits A—F with text.
Wrong: Treating A as a letter.

Right: In hexadecimal, A=10, B=11, C=12, D=13, E=14, F=15. These are numbers, not letters.

Pitfall 8 — Getting place values wrong.
Wrong: Starting binary place values from 1 on the left.
Right: Binary place values from left to right are: 128, 64, 32, 16, 8, 4, 2, 1. Memorise this!

Pitfall 9 — Thinking shifts work for any multiplication.
Wrong: "A binary shift can multiply by 3."
Right: Binary shifts only multiply/divide by powers of 2 (2, 4, 8, 16, etc.).

Pitfall 10 — Not checking answers.
Wrong: Submitting without verification.
Right: Always check by converting back or adding place values. A few seconds of checking can save marks.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.